

Σύστημα Διαχείρισης Παραγγελιών + Δίκτυο 5 παραρτημάτων

Απαλλακτική Εργασία — Τεχνολογία Διαδικτύου στην Ψηφιακή Βιομηχανία

Ελένη Βασιλική Διαμάντη — Α.Μ. 19389308

Διδάσκων: Δρ. Μιχάλης Ξευγένης · 4 Ιουνίου 2026

Στόχος εργασίας

- 1. Διαδικτυακή εφαρμογή PHP/MariaDB – XAMPP, CRUD, 7 σελίδες, 4 πίνακες
- 2. Δικτυακή τοπολογία 5 παραρτημάτων – Cisco Packet Tracer, OSPF, VLANs, ACLs

Σενάριο εφαρμογής

- Μικρή επιχείρηση ηλεκτρονικών ειδών
- Διαχείριση: πελάτες → προϊόντα → παραγγελίες → γραμμές
- Snapshot τιμής στη γραμμή παραγγελίας (ιστορικό)
- Χρήστης: υπάλληλος γραφείου (back-office)

The screenshot displays the PAPI-net web application interface. The browser address bar shows the URL `localhost:8080/papinet/public/index.php`. The application header includes the logo "PAPI-net" and the title "Διαχείριση Παραγγελιών", along with navigation links: Αρχική, Πελάτες, Προϊόντα, and Παραγγελίες.

The main content area features a section titled "Καλωσορίσατε" (Welcome) with a subtitle: "Αυτή είναι η αρχική σελίδα του συστήματος διαχείρισης παραγγελιών PAPI-net." Below this, there are four summary cards:

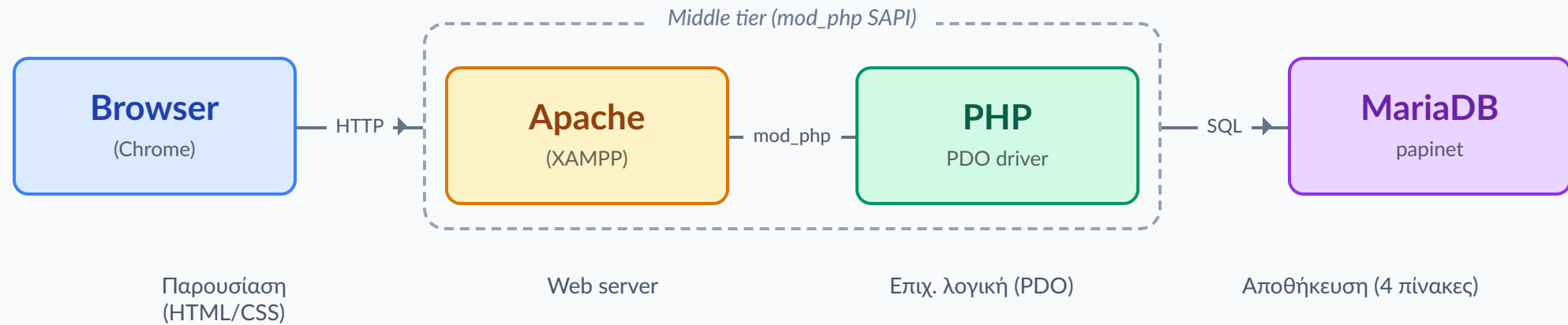
- 6 Πελάτες (Customers)
- 8 Προϊόντα (Products)
- 7 Παραγγελίες (Orders)
- 494,77 € Έσοδα (Revenue)

Below the summary cards, there is a section titled "Τελευταίες 5 παραγγελίες" (Last 5 orders) containing a table with the following data:

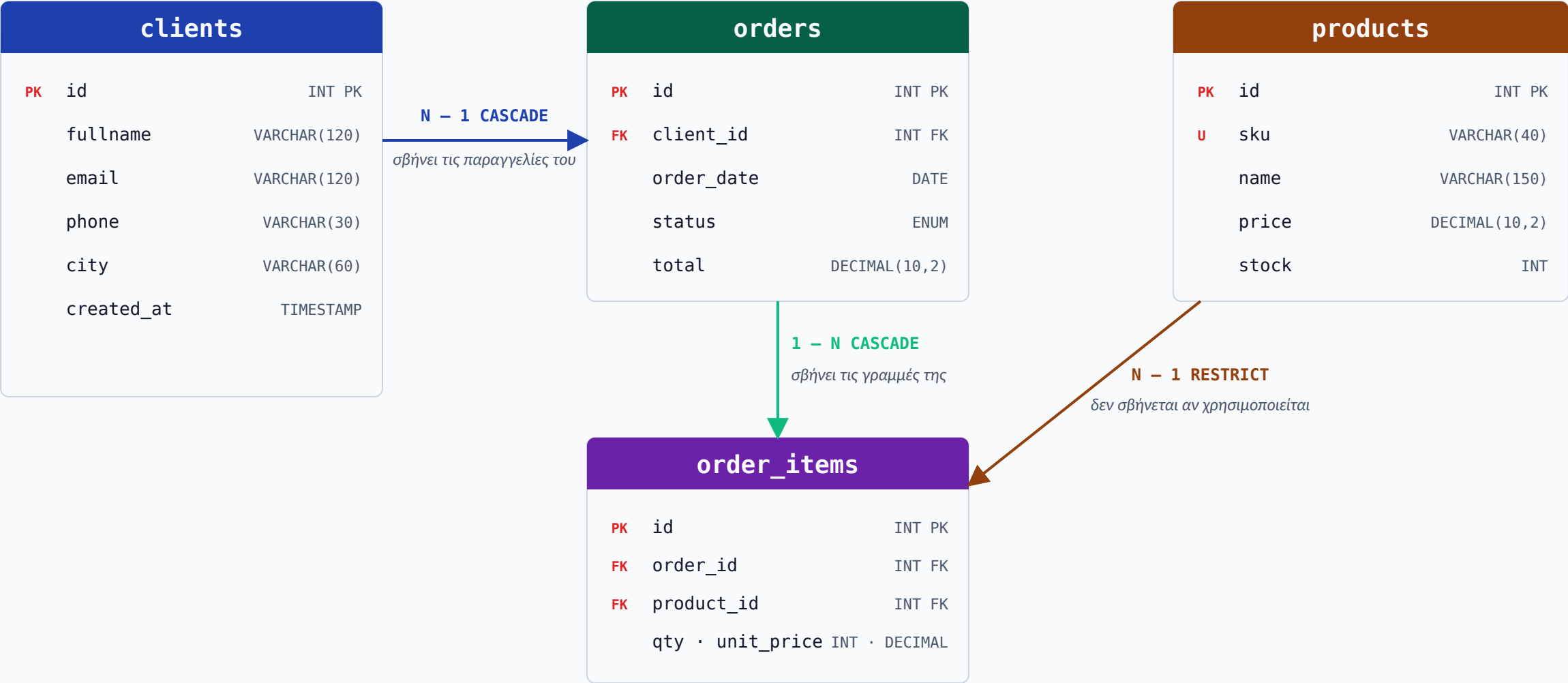
#	Πελάτης	Ημερομηνία	Κατάσταση	
7	Εμμανουήλ Μανος	2026-05-19	NEW	Άνοιγμα
6	Γεώργιος Παπαδόπουλος	2026-05-19	NEW	Άνοιγμα
5	Γεώργιος Παπαδόπουλος	2026-05-10	SHIPPED	Άνοιγμα
4	Νικόλαος Καραγιάννης	2026-05-05	PAID	Άνοιγμα
3	Ελένη Διαμάντη	2026-05-01	NEW	Άνοιγμα

At the bottom of the page, a footer contains the copyright notice: "© 2026 PAPI-net — Εκπαιδευτική εφαρμογή (Απολλακτική, Διαμάντη Ελένη Βασιλική, AM 19389308)".

Αρχιτεκτονική 3-tier (client-server)



Βάση δεδομένων (ER)



Δείγμα Σύνδεσης Βάσης Δεδομένων

Αρχείο `includes/config.php` – σύνδεση PHP ↔ MariaDB μέσω PDO

```
<?php
$host      = 'localhost';
$db        = 'papinet';
$user      = 'root';
$pass      = ''; // Κενό για προεπιλεγμένο χρήστη XAMPP
$charset   = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES  => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
    throw new \PDOException($e->getMessage(), (int)$e->getCode());
}
?>
```

Γιατί αυτές οι ρυθμίσεις

utf8mb4 Πλήρης ελληνικά + emoji

Το MySQL `utf8` είναι μόνο 3 bytes – σπάει σε ☹ και σπάνιους χαρακτήρες. Το `utf8mb4` είναι το πραγματικό 4-byte UTF-8. Σήμερα πάντα αυτό.

ERRMODE_EXCEPTION Άμεση ανίχνευση σφαλμάτων

Default: σπασμένο query επιστρέφει `false` σιωπηλά → ώρες debugging. Εδώ κάθε failure γίνεται `PDOException` που πιάνεις με `try/catch`.

FETCH_ASSOC Καθαρό associative array

Default `FETCH_BOTH`: κάθε πεδίο 2 φορές (όνομα + index) → διπλάσιο memory. Εδώ μόνο `$row['name']` – γρήγορο, καθαρό.

EMULATE_PREPARES = false Real prepared statements (anti-injection)

Default: η PHP κάνει fake escaping. Με `false`, query + params πάνε χωριστά στον MySQL server → προστασία στο πρωτόκολλο + σωστά types (int μένει int).

Δομή Καταλόγων & Αρχείων

Πραγματική δομή `github.com/idre19389308/papi-net` — webapp + report + network configs

```
papi-net/
├── README.md           # Επισκόπηση project
├── final-report/       # Αναφορά + interactive diagrams
├── packet-tracer/     # Configs 5 routers + 5 switches
├── presentation/      # Slides + outline
├── webapp/            # PHP/MariaDB application
│   ├── README.md
│   ├── includes/
│   │   ├── config.php  # PDO σύνδεση + ρυθμίσεις
│   │   ├── header.php  # Κεφαλίδα σελίδων
│   │   └── footer.php  # Υποσέλιδο σελίδων
│   ├── public/        # DocumentRoot του Apache
│   │   ├── index.php   # Dashboard
│   │   ├── clients.php # Λίστα πελατών
│   │   ├── clients_edit.php
│   │   ├── products.php
│   │   ├── products_edit.php
│   │   ├── orders.php
│   │   ├── orders_view.php
│   │   └── assets/
│   │       └── style.css # Custom CSS (no Bootstrap)
│   ├── sql/
│   │   ├── schema.sql  # DDL: 4 πίνακες InnoDB + FK
│   │   └── seed.sql    # Sample data
│   └── docs/           # Screenshots + ER text
```

Βασικά Αρχεία & Ρόλοι

Καθαρός διαχωρισμός: **public/** = ό,τι σερβίρει ο Apache, **includes/** = ιδιωτικός κώδικας, **sql/** = schema.

includes/config.php

Δημιουργεί `$pdo` (PDO connection) + κοινές σταθερές. Required από κάθε σελίδα.

includes/header.php / footer.php

Κοινό HTML (nav, logo, footer) — αποφυγή επανάληψης κώδικα.

public/index.php

Dashboard — σύνολα + τελευταίες 5 παραγγελίες.

public/{clients,products,orders}.php

Λίστες CRUD με αναζήτηση + φόρμα προσθήκης.

public/*_edit.php

Update / Delete με CASCADE ή RESTRICT semantics.

sql/schema.sql + seed.sql

One-shot setup της βάσης στο XAMPP (import σε phpMyAdmin).

docs/

Screenshots + ER text για τεκμηρίωση.

Δυναμικές σελίδες (CRUD)

7 σελίδες με πλήρη CRUD λειτουργίες · prepared statements παντού · κοινό header/footer

index.php

Dashboard — σύνολα πελατών, προϊόντων, παραγγελιών, εσόδων + τελευταίες 5 παραγγελίες

R

clients.php

Λίστα πελατών με αναζήτηση (LIKE fullname/email, παράμετρος ?q=) + φόρμα προσθήκης

C R

clients_edit.php

Επεξεργασία / διαγραφή πελάτη — ON DELETE CASCADE διαγράφει και τις παραγγελίες του

U D

products.php

Κατάλογος προϊόντων με τιμές & απόθεμα + προσθήκη (UNIQUE constraint στο SKU)

C R

products_edit.php

Επεξεργασία / διαγραφή προϊόντος — RESTRICT αν υπάρχει σε ιστορικές παραγγελίες

U D

orders.php

Λίστα παραγγελιών με φίλτρο κατάστασης (? status=) + whitelist guard + JOIN clients

C R D

orders_view.php

Λεπτομέρειες παραγγελίας — προσθήκη/αφαίρεση γραμμών, αλλαγή κατάστασης, υπολογισμός συνόλου

R U D

Ασφάλεια

Prepared LIKE wildcards · whitelist guard (ALL/NEW/PAID/SHIPPED/CANCELLED) · anti-injection σε όλα τα POST

Pipeline Δημιουργίας Εγγραφής

Παράδειγμα: ο χρήστης προσθέτει πελάτη — τι συμβαίνει σε κάθε στάδιο



SCHEMA · DDL (ΜΙΑ ΦΟΡΑ, SETUP)

```
CREATE DATABASE papinet
  DEFAULT CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY,
  fullname VARCHAR(120) NOT NULL,
  email VARCHAR(120),
  phone VARCHAR(30),
  city VARCHAR(60),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB CHARSET=utf8mb4;
```

INSERT · ΕΝΑ ΝΕΟ ΠΕΛΑΤΗ (ΚΑΘΕ ΦΟΡΑ)

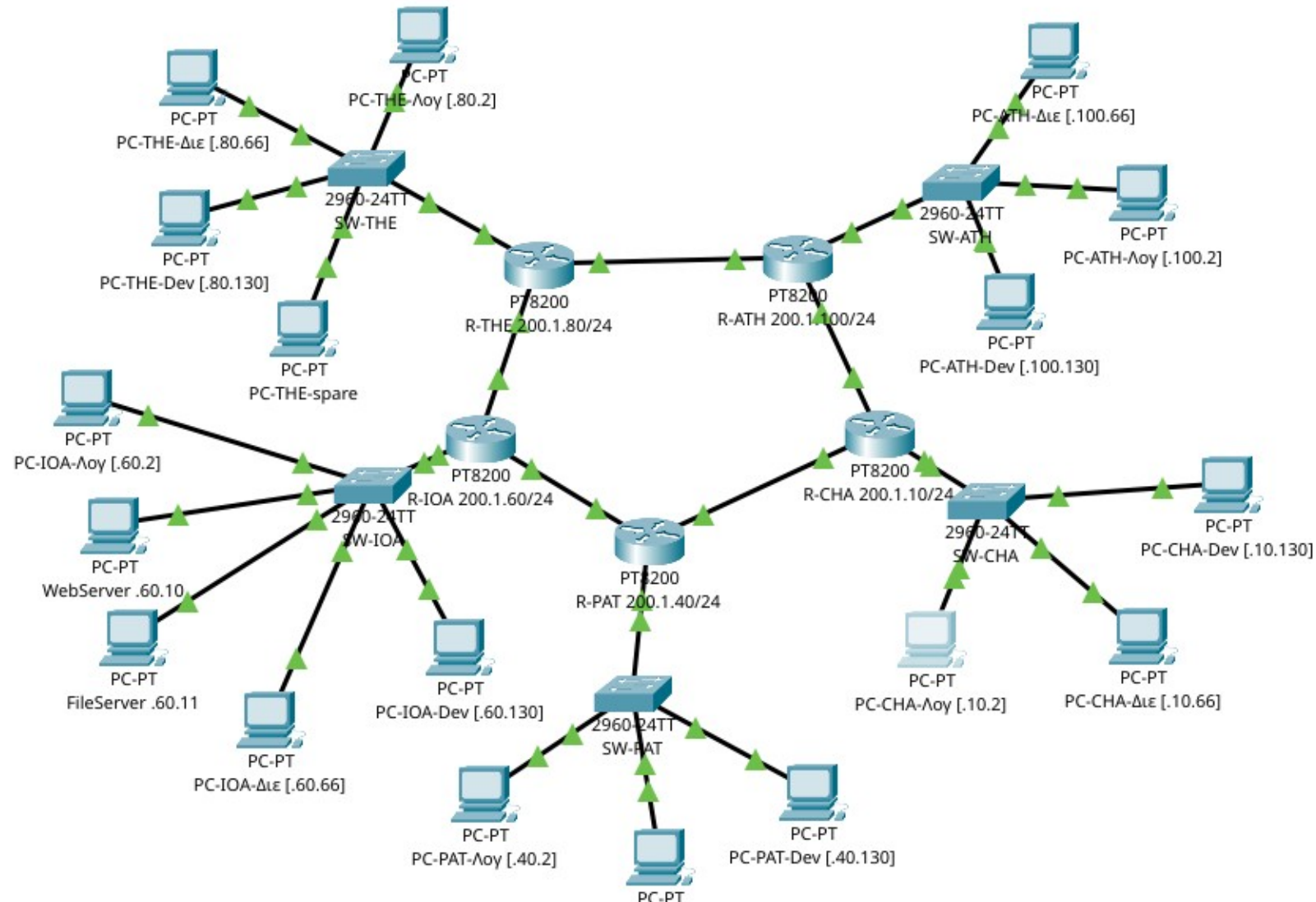
```
// PHP - στο clients_edit.php
$stmt = $pdo->prepare(
  "INSERT INTO clients
    (fullname, email, phone, city)
  VALUES (?, ?, ?, ?)"
);

$stmt->execute([
  $_POST['fullname'],
  $_POST['email'],
  $_POST['phone'],
  $_POST['city'],
]);

$newId = $pdo->lastInsertId();
```

Δικτυακή τοπολογία

Logical Physical x: 964, y: 462



Σχέδιο διευθυνσιοδότησης

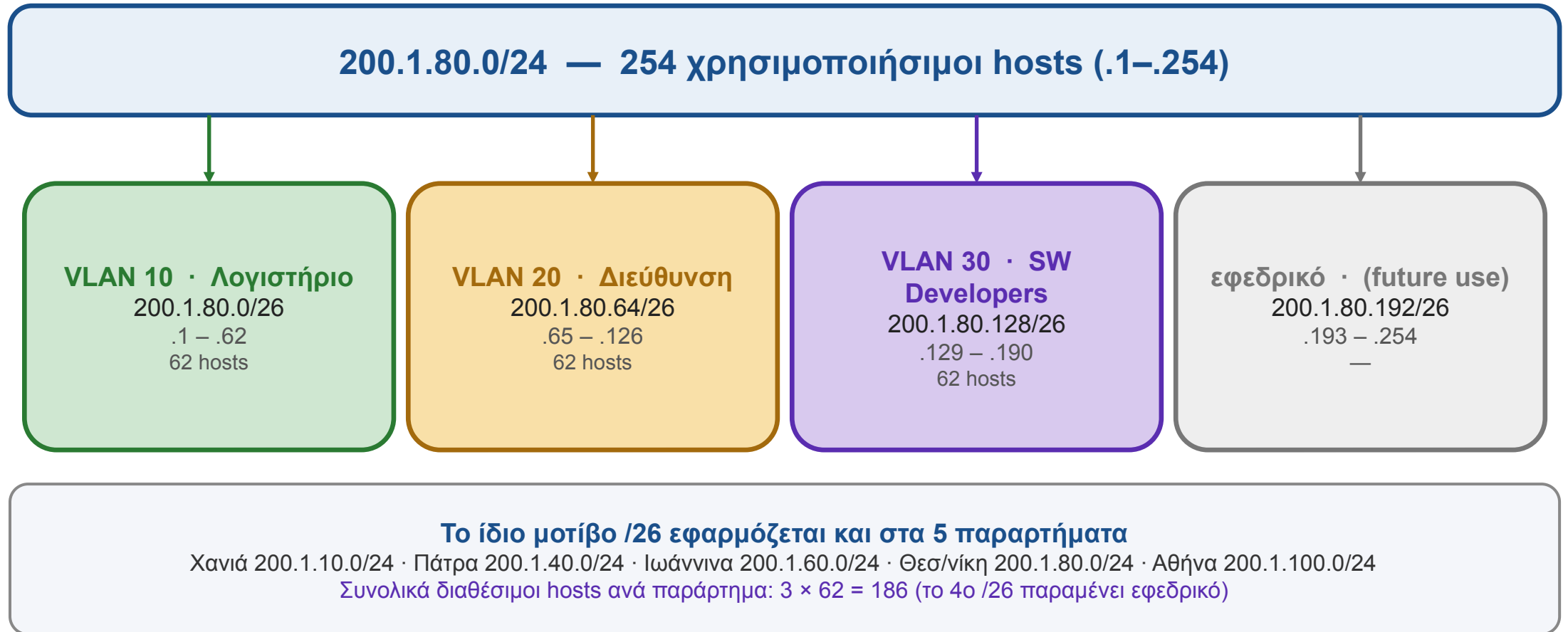
LAN /24 ανά παράρτημα

- Θεσσαλονίκη — 200.1.80.0 / 24
- Ιωάννινα — 200.1.60.0 / 24
- Πάτρα — 200.1.40.0 / 24
- Αθήνα — 200.1.100.0 / 24
- Χανιά — 200.1.10.0 / 24

WAN /30 ανά link

- THE IOA — 200.1.70.0 / 30
- IOA PAT — 200.1.50.0 / 30
- THE ATH — 200.1.90.0 / 30
- ATH CHA — 200.1.20.0 / 30
- PAT CHA — 200.1.30.0 / 30

VLANs (3 ανά παράρτημα)



VLAN 10 Λογιστήριο · VLAN 20 Διεύθυνση · VLAN 30 SW Developers · /26 ανά VLAN (62 hosts)

OSPF δυναμική δρομολόγηση

Πώς οι routers «μαθαίνουν» μόνοι τους το δίκτυο και βρίσκουν τον βέλτιστο δρόμο

1



Hello

Κάθε router στέλνει **Hello packets** για να βρει τους άμεσους γείτονές του.

```
multicast 224.0.0.5
```

2



LSA Exchange

Όλοι ανταλλάσσουν **Link-State Advertisements**: «έχω αυτά τα δίκτυα με αυτά τα costs».

```
LSA flooding
```

3



LSDB

Κάθε router χτίζει **πλήρη χάρτη** του δικτύου (Link-State Database) — όλοι βλέπουν το ίδιο.

```
show ip ospf database
```

4



SPF / Dijkstra

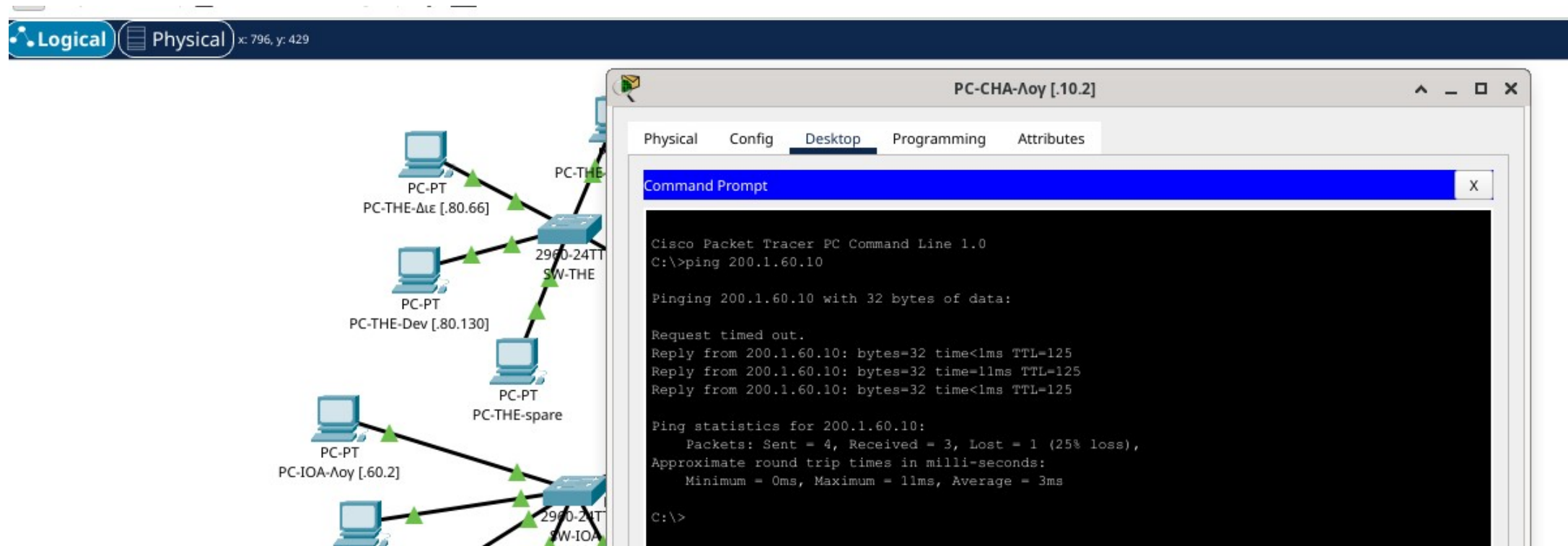
Τρέχει αλγόριθμο Dijkstra → υπολογίζει **συντομότερη διαδρομή** προς κάθε δίκτυο.

```
show ip route ospf
```

✂ **Αν πέσει link:** Ο γείτονας ανιχνεύει την απώλεια **μέσα σε δευτερόλεπτα**, στέλνει LSA update → όλοι επανυπολογίζουν αυτόματα → η κίνηση παίρνει **εναλλακτικό δρόμο γύρω από τον δακτύλιο**. Στο PAPI-net: 5 routers, area 0, αυτόματη ανθεκτικότητα χωρίς χειροκίνητη παρέμβαση.

Inter-VLAN routing + ACLs

- **Router-on-a-stick** — μία φυσική θύρα Gi0/0
- **Subinterfaces** — Gi0/0.10, .20, .30
- **802.1Q tagging** — διαφορετικό VLAN ID ανά subinterface
- **ACL 110 στον R-ATH** — extended numbered ACL
- **deny ip 200.1.100.0 0.0.0.63 200.1.100.128 0.0.0.63**
- **Στόχος:** ο SW Developers (VLAN 30) → ΟΧΙ στο Λογιστήριο (VLAN 10)
- **permit ip any any** — υπόλοιπη κίνηση επιτρέπεται



Απαντήσεις θεωρητικών ερωτημάτων

(α) Υποδίκτυα

10 πριν το VLANing · **20 μετά** το VLANing (3 LAN VLAN ανά παράρτημα + 5 WAN /30)

Υπολογισμός

Πριν: 5 παρ × 1 LAN + 5 WAN = 10 Μετά: 5 παρ × 3 VLAN + 5 WAN = 15 + 5 = **20**
Bits για 20: $2^5 = 32 \geq 20 \rightarrow$ χρειάζονται 5 bits subnet

(β) Πλήθος Hosts

/24 $\rightarrow$ 254 hosts · /26 ανά VLAN $\rightarrow$ 62 hosts × 3 = **186 hosts** ωφέλιμοι/παράρτημα

Υπολογισμός

/24: $2^8 - 2 =$ **254** hosts (-network, -broadcast) /26: $2^6 - 2 =$ **62** hosts ανά VLAN Σύνολο/παράρτημα: $62 \times 3 =$ **186** WAN /30: $2^2 - 2 =$ 2 hosts (point-to-point)

(γ) Θεσσαλονίκη 200.1.80.0/24

Δίκτυο: **200.1.80.0** · Broadcast: **200.1.80.255** · hosts: .1 - .254

Υπολογισμός

Μάσκα /24 = 255.255.255.0 = 11111111.11111111.11111111.00000000 Host bits = 8 $\rightarrow$ $2^8 - 2 =$ **254** χρήσιμοι Network: όλα τα host bits 0 $\rightarrow$.0 Broadcast: όλα τα host bits 1 $\rightarrow$.255

Ευχαριστώ — Ερωτήσεις

Σύνοψη παραδοτέων

- **Web App** — PHP/MariaDB, 7 σελίδες CRUD, PDO, prepared statements
- **Δίκτυο** — 5 παραρτήματα, OSPF area 0, 3 VLAN ανά site, router-on-a-stick
- **Ασφάλεια** — extended ACL για inter-departmental traffic control

 **Αποθετήριο** <https://github.com/idpe19389308/papi-net>